

Sisteme de Operare. Laborator 4

Honorius Gâlmeanu
galmeanu@unitbv.ro

March 12, 2023

1 Funcțiile stat, fstat și lstat

Permit obținerea informațiilor referitoare la fișiere.

```
#include <sys/stat.h>
```

```
int stat (const char *pathname, struct stat *statbuf);  
int fstat(int fd, struct stat *statbuf);  
int lstat(const char *pathname, struct stat *statbuf);
```

Funcțiile vor returna 0 sau -1 în caz de eroare. Funcția `stat` va returna o structură cu informații despre fișierul pasat ca parametru. Funcția `fstat` realizează același lucru dar fișierul trimis ca parametru este reprezentat de file descriptor. Funcția `lstat` este identică cu `stat`, cu excepția fișierelor de tip legătură simbolică. Ea va întoarce informații despre legătura simbolică și nu despre fișierul referit de legătura simbolică.

Toate cele trei funcții vor seta corespunzător membrii variabilei de tip `struct stat` (variabila trebuie să fie deja creată la apelul funcției).

```
struct stat {  
    dev_t      st_dev;          /* ID of device containing file */  
    ino_t      st_ino;          /* Inode number */  
    mode_t     st_mode;         /* File type and mode */  
    nlink_t    st_nlink;        /* Number of hard links */  
    uid_t      st_uid;          /* User ID of owner */  
    gid_t      st_gid;          /* Group ID of owner */  
    dev_t      st_rdev;         /* Device ID (if special file) */  
    off_t      st_size;         /* Total size, in bytes */  
    blksize_t  st_blksize;      /* Block size for filesystem I/O */  
    blkcnt_t   st_blocks;       /* Number of 512B blocks allocated */  
  
    /* Since Linux 2.6, the kernel supports nanosecond  
       precision for the following timestamp fields.  
       For the details before Linux 2.6, see NOTES. */  
  
    struct timespec st_atim;     /* Time of last access */  
    struct timespec st_mtim;     /* Time of last modification */  
    struct timespec st_ctim;     /* Time of last status change */  
  
    #define st_atime st_atim.tv_sec      /* Backward compatibility */  
    #define st_mtime st_mtim.tv_sec  
    #define st_ctime st_ctim.tv_sec  
};
```

2 Tipuri de fișiere

Sistemul Linux definește următoarele tipuri de fișiere, tip specificat de membrul `st_mode`:

1. fișier normal, `S_ISREG()`; acest tip de fișier conține date; nu există distincție între datele de tip text sau cele binare;
2. director, `S_ISDIR()`; conține numele altor fișiere și pointeri spre informațiile definite de acele fișiere; orice proces care are drept de citire din director poate citi conținutul său, dar scrierea se face doar prin intermediul apelurilor sistem;
3. fișier special de tip caracter, `S_ISCHR()`, unde accesul se face liniar (secvențial), caracter cu caracter;
4. fișier special de tip bloc, `S_ISBLK()`, pentru care accesul la date se face în blocuri, specificând adresa blocului;
5. fișier de tip pipe (FIFO), `S_ISFIFO()` utilizat pentru comunicația inter-procese; pentru cele specificate prin nume de fișier, este vorba de un named-pipe;
6. fișier de tip legătură simbolică, `S_ISLNK()`, ce specifică calea unui alt fișier la care se referă;
7. fișier de tip socket, `S_ISSOCK()`, folosit la comunicația între procese prin rețea.

3 Set-User-ID și Set-Group-ID

Fiecare proces are asociate următoarele ID-uri:

- real user ID, real group ID: identifică utilizatorul și grupul din care face parte utilizatorul care a creat procesul. Ele sunt preluate din fișierul `/etc/passwd` la login;
- effective user ID, effective group ID: determină drepturile de acces la fișiere pentru utilizator;
- saved set-user-ID, saved set-group-ID: copii ale ID-urilor efective când un program este executat.

Fiecare fișier are un **owner** și un **owner group**. Ei sunt specificați în membrii `st_uid` și `st_gid` ai structurii `stat`.

La execuția unui proces, effective user ID este același cu real user ID (la fel și pentru grup). Există posibilitatea de a seta un bit din variabila `st_mode` pentru acel fișier, astfel ca la execuția acelui fișier se va seta effective user ID ca fiind owner-ul fișierului (`st_uid`). Un lucru similar se poate face pentru effective group ID. Acești doi biți din `st_mode` se numesc set-user-ID respectiv set-group-ID. De exemplu, dacă owner-ul fișierului este superuser-ul root și fișierul are setat bitul set-user-ID, atunci la rularea sa el va avea drepturile superuser-ului. Programul `passwd` este un program set-user-ID (observați bitul de execuție pentru utilizator):

Command Line

```
[user@fedora lab4]$ ls -al /usr/bin/passwd
-rwsr-xr-x. 1 root root 69608 Jan 21 2022 /usr/bin/passwd
```

4 Drepturile de acces la fișier. Funcția `chmod`

Membrul `st_mode` definește drepturile de acces la fișier. Sunt definite următoarele măști pentru a putea opera cu acești biți (secțiunea 7 din pagina de manual pentru `inode`):

<code>S_ISUID</code>	04000	set-user-ID bit (see <code>execve(2)</code>)
<code>S_ISGID</code>	02000	set-group-ID bit (see below)
<code>S_ISVTX</code>	01000	sticky bit (see below)
<code>S_IRWXU</code>	00700	owner has read, write, and execute permission
<code>S_IRUSR</code>	00400	owner has read permission
<code>S_IWUSR</code>	00200	owner has write permission
<code>S_IXUSR</code>	00100	owner has execute permission

S_IRWXG	00070	group has read, write, and execute permission
S_IRGRP	00040	group has read permission
S_IWGRP	00020	group has write permission
S_IXGRP	00010	group has execute permission
S_IRWXO	00007	others (not in group) have read, write, and execute permission
S_IROTH	00004	others have read permission
S_IWOTH	00002	others have write permission
S_IXOTH	00001	others have execute permission

Acești biți pot fi modificați folosind comanda (și funcția) `chmod`:

```
#include <sys/stat.h>
int chmod(const char *pathname, mode_t mode);
int fchmod(int fd, mode_t mode);
```

Există următoarele reguli de acces:

- pentru a deschide un fișier, trebuie să avem drept de execuție pentru fiecare director unde este menționat numele fișierului. De aceea bitul de execuție pentru directoare se mai numește și **search bit**. Dreptul de citire pentru un director permite obținerea listei fișierelor din director dar nu permite căutarea unui anumit fișier specificând calea;
- nu se poate crea sau șterge un fișier într-un/dintr-un director dacă nu avem drept de execuție și de scriere în acel director;
- pentru a putea citi, scrie sau executa un fișier, avem nevoie de drept de citire, scriere sau execuție pentru acel fișier.

Testele realizate de kernel pentru stabilirea dreptului de acces sunt:

1. dacă effective user ID este 0 (superuser), accesul este permis;
2. dacă effective user ID este egal cu real user ID, i.e. fișierul respectiv este al procesului:
 - (a) accesul este permis dacă biții de acces pentru user sunt setați în conformitate cu tipul de acces;
 - (b) dacă nu, accesul nu este permis;
3. dacă effective group ID al procesului sau unul din saved group ID este egal cu group ID-ul fișierului:
 - (a) accesul este permis dacă biții de acces pentru grup sunt setați în conformitate cu tipul de acces;
 - (b) dacă nu, accesul nu este permis;
4. accesul este permis dacă biții de acces pentru other sunt setați în conformitate cu tipul de acces.

User ID pentru un nou fișier este setat ca fiind effective user ID-ul procesului care îl creează. Group ID pentru un fișier nou creat poate fi effective group ID-ul procesului sau group ID-ul directorului în care este creat. Se poate alege una din metode prin setarea set-group-ID, sau printr-un flag al comenzii `mount`.

Un proces poate modifica biții de acces pentru un fișier doar dacă effective user ID-ul procesului este egal cu cel al owner-ului fișierului sau dacă procesul are drept de superuser.

Bitul `S_ISVTX` este bitul denumit bit-saved text și este referit cu numele de sticky-bit. Dacă saved-text bit este setat pentru un director, atunci un fișier din respectivul director poate fi șters sau redenumit numai dacă utilizatorul are drept la scriere în acel director și este îndeplinită una din condițiile:

- user-ul este owner-ul fișierului;
- user-ul este owner-ul directorului sau este superuser.

Sticky-bit se utilizează în cazul directorului `/tmp`. Orice user are drept de scriere, citire și execuție în acest director, dar nu se permite ștergerea sau redenumirea fișierelor deținute de alți utilizatori.

5 Funcția access

Funcția verifică dacă procesul din care este apelată poate accesa un fișier (pentru citire, scriere, execuție):

```
#include <unistd.h>
int access(const char *pathname, int mode);
```

Parametrul mode indică ce verificare se face asupra fișierului, astfel:

- R_OK, testează dreptul la citire;
- W_OK, testează dreptul la scriere;
- X_OK, testează dreptul la execuție;
- F_OK, testează existența fișierului;
- X_OK, este o mască ce le cuprinde pe toate cele anterioare.

6 Funcția umask

La crearea unui fișier sau director, se utilizează o mască pentru drepturile de acces. Mai specific, drepturile specificate de mască sunt resetate (puse pe 0). Această mască se setează cu ajutorul funcției (și comenzii) umask:

```
#include <sys/stat.h>
mode_t umask(mode_t mask);
```

Masca va fi specificată cu ajutorul măștilor specificate pentru variabila st_mode (S_IRUSR, S_IWUSR etc.)

7 Funcția chown

User ID-ul sau group ID-ul unui fișier se modifică folosind:

```
#include <unistd.h>

int chown(const char *pathname, uid_t owner, gid_t group);
int fchown(int fd, uid_t owner, gid_t group);
int lchown(const char *pathname, uid_t owner, gid_t group);
```

În cazul sistemelor BSD schimbarea ownership-ului este permisă doar superuser-ului; se previne astfel atribuirea fișierelor către alți utilizatori cu scopul sustragerii de la aplicarea restricțiilor quota.

8 Exemple

Pentru a putea compila programele date ca exemplu va fi necesar ca mai întâi să se construiască biblioteca liblab4.a. Acesta se face cu comanda:

Command Line

```
make lib
```

După generarea bibliotecii liblab4.a, se compilează exemplele cu:

Command Line

```
make
```

8.1 Programul filetype

Programul tipărește tipul fișierelor ale căror căi sunt transmise în linia de comandă. Se exemplifică utilizarea funcției `lstat`. Programul se execută astfel:

Command Line

```
$ ./filetype ./filetype /etc /dev/tty /dev/sr0 /var/run
```

8.2 Programul access

Programul verifică drepturile de acces și posibilitatea de citire a unui fișier. Se exemplifică utilizarea funcției `access`:

Command Line

```
$ ls -l access
$ ./access access
```

8.3 Programul umask

Programul creează fișierele `foo` și `bar` folosind măști diferite pentru drepturile de acces. Se exemplifică utilizarea funcției `umask`. Se va executa astfel:

Command Line

```
$ umask
$ ./umask
$ ls -l foo bar
```

8.4 Programul changemod

Programul schimbă drepturile de acces pentru fișierele `foo` și `bar` create de programul anterior. Se exemplifică utilizarea funcției `chmod`. Se va executa astfel:

Command Line

```
$ ls -l foo bar
$ ./changemod
$ ls -l foo bar
```

9 Exerciții

Exercițiul 1

Rulați toate exercițiile prezentate și încercați să înțelegeți cum lucrează. Vedeți paginile de manual pentru funcțiile utilizate.

Exercițiul 2

Modificați programul `filetype`, înlocuind funcția `lstat` cu funcția `stat`. Ce se va întâmpla dacă se execută programul cu un argument de tip legătură simbolică?

Exercițiul 3

Modificați programul `umask` în așa fel încât să nu se mai cheme apelul de sistem `umask`; spargeți programul în două programe, `umask1` și `umask2`, care se ocupă separat de cele două fișiere. Realizați același efect folosind doar comanda `bash umask`, înaintea apelării fiecăruia.

Exercițiul 4

Creați un program C care modifică bitul set-user-ID al unuia din executabilele folosite la laborator, unde numele acestuia este trimis ca parametru.